

**SIMATS ENGINEERING
ASSESSMENT TOOL 2**

COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation. CO (BL4 , BL5)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

**Annexure A
CASE STUDY**

Code of Conduct:

I U.Divya/192372277 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Divya

Annexure A

1. Weak Password Hashing in Web Application

A web application stores user passwords using unsalted SHA-1 hashes. After a breach, attackers quickly crack multiple passwords using precomputed tables. Analyze how the lack of salting and use of SHA-1 contributed to this vulnerability. Recommend a secure password hashing strategy and justify your choice.

Case Analysis

The web application stores passwords using **unsalted SHA-1 hashes**. This creates two major security weaknesses:

- **SHA-1 is a fast general-purpose hash**, so attackers can test a very large number of password guesses quickly.
- Because **no salt is used**, the same password always produces the same SHA-1 hash.
- Attackers can use **precomputed rainbow tables** containing common password-to-hash mappings.
- If many users have the same password, their identical hashes immediately reveal that they share a password.
- Once an attacker obtains the password database, offline password cracking can be performed without interacting with the application.

Security Impact

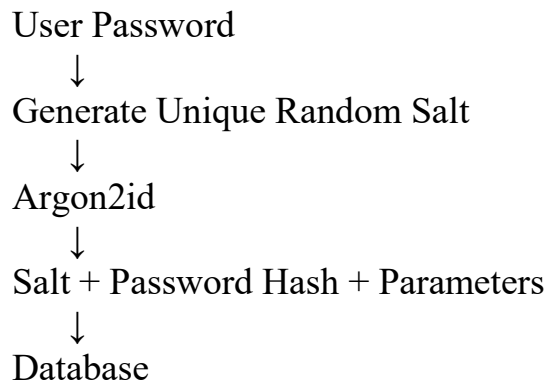
The breach can result in:

1. **Password disclosure** through offline cracking.
2. **Credential stuffing** against other websites when users reuse passwords.
3. **Account takeover** of multiple users.
4. Exposure of sensitive user information.
5. Loss of customer confidence and reputation.

Recommended Solution

The application should replace SHA-1 with a **password-specific adaptive hashing algorithm**, preferably **Argon2id**. bcrypt or scrypt are also appropriate choices.

A secure password storage process should be:



Each user should have a **unique random salt**, and the algorithm should use deliberately expensive computation and memory requirements.

Justification

Argon2id is preferable because it is specifically designed for password hashing and is resistant to large-scale password cracking, including attacks using specialized hardware. Unlike SHA-1, its computational and memory cost can be configured to make offline guessing substantially more expensive.

Conclusion: The vulnerability occurred because a fast, obsolete hash was used without salting. Moving to Argon2id with a unique salt for every password significantly improves password security.

2. Digital Signature Compromise in Financial System

A financial institution uses digital signatures for transaction approval. An attacker gains access to a user's private key and performs unauthorized transactions. Evaluate how the compromise affects authentication and non-repudiation. Propose mitigation strategies to secure private keys and prevent such incidents.

Case Analysis

The financial institution uses digital signatures to authorize transactions. An attacker has obtained a user's **private signing key**.

A digital signature normally provides:

- **Authentication** – identifies the holder of the signing key.
- **Integrity** – detects modification of the signed transaction.
- **Non-repudiation** – provides evidence that a transaction was signed by the corresponding private key, subject to the legal and operational framework.

However, once the private key is compromised, an attacker can generate **valid-looking signatures**.

Security Impact

1. Authentication is compromised

The attacker can impersonate the legitimate user because possession of the private key allows the attacker to create signatures that verify against the user's public key.

2. Non-repudiation is weakened

The institution may see a mathematically valid signature, but it can no longer safely assume that the legitimate user actually authorized the transaction.

3. Financial loss

The attacker may authorize:

- Unauthorized fund transfers
- False payments
- Account changes
- Fraudulent financial transactions

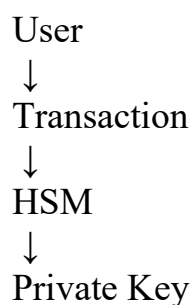
4. Trust is affected

Transactions signed during the period of key compromise may require investigation and possible reversal.

Recommended Mitigation Strategies

1. Protect private keys using HSMs

Store sensitive signing keys in a **Hardware Security Module (HSM)** rather than ordinary files.



↓
Digital Signature

The private key should ideally never be exportable from the HSM.

2. Use strong authentication

Require MFA before sensitive transactions are approved.

3. Implement transaction signing

Instead of simply signing a login request, sign transaction-specific information such as:

Account + Amount + Recipient + Timestamp + Transaction ID

This prevents a valid signature from being reused for a different transaction.

4. Revoke the compromised key

Immediately revoke the corresponding certificate and prevent further use of the compromised private key.

5. Issue a new key pair

Generate a new private/public key pair and issue a new certificate.

6. Monitor transactions

Use fraud detection and anomaly monitoring to identify unusual transactions.

7. Use key rotation

Regularly replace cryptographic keys according to organizational policy.

Justification

Private-key protection is critical because **digital signatures are only as trustworthy as the private key that creates them**. HSMs, MFA, transaction-specific signing, certificate revocation, and monitoring significantly reduce the probability and impact of private-key compromise.

Conclusion: The compromise destroys confidence in signatures created with the affected key during the compromise period. Immediate key revocation, secure key

storage, re-keying, and transaction monitoring are essential.

3. Kerberos Authentication Failure in Distributed System

A distributed system using Kerberos experiences frequent authentication failures due to time synchronization issues between servers.

Analyze how time synchronization impacts Kerberos functionality. Suggest solutions to ensure reliable authentication and maintain system security.

Case Analysis

The distributed system uses **Kerberos**, which relies heavily on synchronized clocks.

Kerberos tickets contain timestamps and have a limited validity period. A server checks whether the timestamp in an authentication request is within an acceptable time difference from its own clock.

For example:

Client Time : 10:00
Kerberos Server : 10:02
Application Server: 10:07

If the difference becomes greater than the configured clock-skew tolerance, authentication may fail.

Why Time Synchronization Matters

Kerberos uses timestamps to:

- Determine whether tickets are still valid.
- Prevent replay of old authentication messages.
- Establish freshness of authentication requests.
- Limit the lifetime of tickets.

Therefore, incorrect system clocks can cause:

Valid Ticket



Timestamp Checked



Clock Difference Too Large



Authentication Rejected

Security Impact

Poor time synchronization can cause:

1. Frequent authentication failures.
2. Users being unable to access services.
3. Expired or not-yet-valid tickets being rejected.
4. Service-to-service authentication failures.
5. Operational downtime.

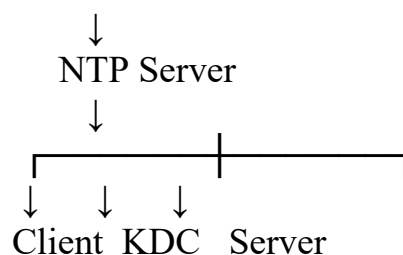
At the same time, simply allowing a very large clock-skew window is not a good solution because it can increase the opportunity for **replay attacks**.

Recommended Solutions

1. Use Network Time Protocol (NTP)

Configure all Kerberos servers, domain systems, and clients to synchronize with trusted time sources.

Trusted Time Source



2. Use redundant time sources

Multiple trusted NTP sources can improve availability.

3. Monitor clock drift

Administrators should monitor systems and generate alerts when clock differences exceed an acceptable threshold.

4. Configure an appropriate clock-skew tolerance

A reasonable tolerance can accommodate normal network and clock variations without unnecessarily weakening replay protection.

5. Ensure reliable KDC infrastructure

Kerberos Key Distribution Centers should be highly available and correctly synchronized.

Justification

Time synchronization is fundamental to Kerberos because timestamps provide both **ticket validity control and replay protection**. Reliable NTP configuration and monitoring provide authentication reliability while preserving the security benefits of short ticket lifetimes and controlled clock skew.

Conclusion: The authentication failures are primarily caused by inconsistent clocks. Proper NTP configuration, monitoring, redundancy, and appropriate clock-skew settings can restore reliable Kerberos authentication.

4. Certificate Authority Breach in PKI System

A Certificate Authority (CA) is compromised, leading to the issuance of fake X.509 certificates for trusted domains.

Evaluate the impact of this breach on system trust and secure communication.

Propose mechanisms to detect and mitigate such attacks in a PKI environment.

Case Analysis

The Certificate Authority (CA) is a trusted entity in a **Public Key Infrastructure (PKI)**. It is responsible for issuing certificates that bind identities, such as domain names, to public keys.

In this case, the CA has been compromised and attackers have obtained the ability to issue fake certificates for trusted domains.

For example:

Legitimate Domain



Legitimate Certificate



Trusted CA

An attacker may instead obtain:

Trusted Domain



Fake Certificate



Compromised CA

Because clients trust the CA, they may incorrectly trust the attacker's certificate.

Security Impact

1. Loss of trust

Browsers and systems may accept fake certificates because they chain back to a trusted CA.

2. Man-in-the-Middle attacks

An attacker may impersonate a legitimate website and intercept encrypted communication.

User



Attacker



Fake Certificate



Trusted CA

3. Data exposure

Sensitive information such as:

- Login credentials
- Financial information
- Session information
- Confidential communications

could potentially be exposed.

4. Loss of PKI trust

The breach affects not only one certificate but potentially many certificates issued by the compromised CA.

Recommended Mitigation

1. Revoke fraudulent certificates

Identify and revoke certificates issued improperly.

2. Revoke or distrust the compromised CA

If the CA itself can no longer be trusted, affected trust anchors/intermediate certificates must be removed or distrusted through appropriate trust-store and ecosystem processes.

3. Certificate Transparency

Use **Certificate Transparency (CT)** logs to detect unexpected certificates issued for domains.

For example:

Certificate Issued



CT Log



Domain Owner Monitoring



Unexpected Certificate Detected



Investigation

4. Certificate revocation checking

Use mechanisms such as:

- OCSP
- CRLs

to determine whether certificates have been revoked.

5. Protect CA private keys

CA signing keys should be strongly protected, ideally using HSMs and strict access controls.

6. Use CA key rotation and incident response

After compromise, affected keys and certificates must be replaced according to a documented incident-response procedure.

Justification

The CA is a **trust anchor** in PKI. If its signing capability is compromised, attackers can potentially create certificates that appear legitimate. Certificate Transparency, revocation, strong CA key protection, monitoring, and rapid distrust/replacement mechanisms help detect and contain fraudulent certificates.

Conclusion: A CA breach can undermine trust in secure communication across many domains. Rapid certificate detection, revocation, CA distrust where necessary, and strong protection of CA keys are essential.

5. Integrity Verification in Cloud Data Storage

A cloud service provider uses hashing to verify file integrity, but users report data tampering incidents. Investigation reveals improper hash validation practices. Analyze how incorrect implementation of hashing can lead to integrity failures. Recommend a robust integrity verification mechanism and justify its effectiveness.

Case Analysis

The cloud provider uses hashing to verify file integrity, but incorrect hash validation practices have allowed tampering incidents.

A cryptographic hash produces a fixed-size value from a file:

Original File



SHA-256



Hash Value

If the file changes:

Modified File



SHA-256



Different Hash

However, hashing alone does not automatically guarantee integrity if the **reference hash itself is not protected**.

How Improper Hash Validation Causes Failure

1. Hash stored with the file

Suppose the system stores:

File = document.pdf

Hash = SHA256(document.pdf)

If an attacker can modify both the file and its stored hash:

Original File → Modified File

Original Hash → Modified Hash

The verification system may incorrectly conclude that the file is valid.

2. Weak or inappropriate hashing

Using outdated or unsuitable algorithms can create security problems.

3. Incorrect comparison

Implementation errors in calculating or comparing the digest can result in false validation.

4. Hash calculated from the wrong data

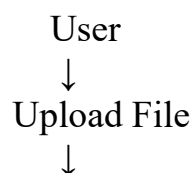
If the application does not hash the exact file contents consistently, changes may go undetected.

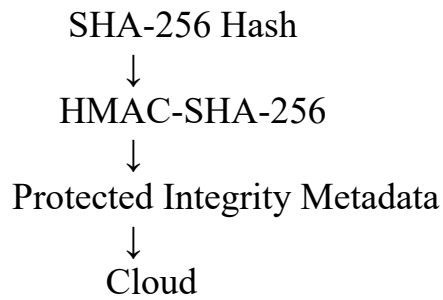
Recommended Solution

Use **SHA-256** or **SHA-3** to calculate the file digest and protect the reference value.

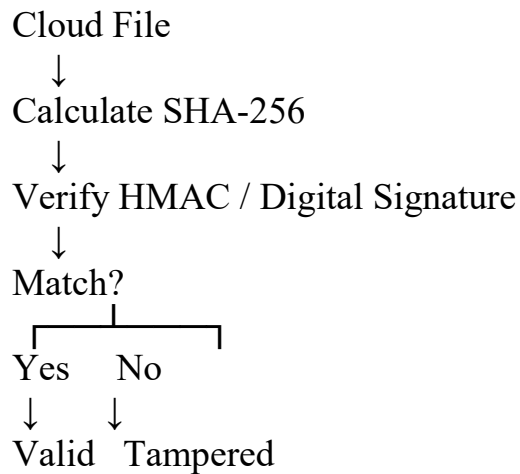
For stronger protection against an untrusted cloud provider, use a **keyed integrity mechanism such as HMAC-SHA-256**, or digitally sign the file/hash with a trusted private key.

A robust design is:





During download:



Recommended HMAC-Based Approach

The user can maintain a secret key and calculate:

$\text{HMAC} = \text{HMAC-SHA-256}(\text{secret key}, \text{file})$

If an attacker modifies the file but does not possess the secret key, they cannot generate a valid replacement HMAC.

Additional Security Measures

- Use TLS for data transmission.
- Maintain version history.
- Use access control and least privilege.
- Maintain audit logs.
- Use immutable or write-protected backups where appropriate.
- Verify files after upload and before download.
- Use digital signatures when non-repudiation is required.

Justification

A plain hash detects accidental or unauthorized changes **only when the trusted reference hash cannot itself be manipulated**. HMAC provides stronger protection when the verifier and cloud storage do not fully trust the storage provider because the attacker needs the secret key to create a valid authentication value.

Conclusion: The cloud integrity failure occurred because the hash validation process did not adequately protect the reference value. Using SHA-256 together with **HMAC-SHA-256 or digital signatures**, secure metadata storage, access control, and auditing provides a much stronger integrity-verification mechanism.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand